

Obliczenia Naukowe

Rafał Wochna 279752

1 Zadanie 1

1.1 Cel zadania

Napisać funkcję, która na podstawie zadanych węzłów $x_0, x_1, \dots, x_n$ i odpowiadających im wartości funkcji $f(x_0), f(x_1), \dots, f(x_n)$ oblicza **ilorazy różnicowe** (współczynniki wielomianu interpolacyjnego Newtona):

$$c_0 = f[x_0], \quad c_1 = f[x_0, x_1], \quad c_2 = f[x_0, x_1, x_2], \quad \dots, \quad c_n = f[x_0, \dots, x_n]$$

1.2 Rozwiązanie

Aby obliczyć ilorazy różnicowe bez użycia tablicy dwuwymiarowej, wykorzystujemy jednowymiarowy wektor `fx`, który początkowo jest kopią wektora wartości funkcji `f`. W kolejnych krokach nadpisujemy go wartościami ilorazów różnicowych wyższych rzędów przez co po każdym wykonaniu będziemy mieli dostępne ilorazy do obliczenia następnych.

Algorytm polega na iteracyjnym obliczaniu ilorazów różnicowych według wzoru:

$$f[x_i, \dots, x_{i+j}] = \frac{f[x_i, \dots, x_{i+j-1}] - f[x_{i+1}, \dots, x_{i+j}]}{x_i - x_{i+j}}.$$

Algorithm 1: `ilorazyRoznicowe(x::VectorFloat64, f::VectorFloat64)`

```
1  $fx \leftarrow f$ 
2 for  $j \leftarrow 1$  to  $n$  do
3   for  $i \leftarrow n$  downto  $j$  do
4      $fx[i] \leftarrow \frac{fx[i] - fx[i-1]}{x[i] - x[i-j]}$ ;
5   end
6 end
7 return  $fx$ ;
```

2 Zadanie 2

2.1 Cel zadania

Napisać funkcję obliczającą wartość wielomianu interpolacyjnego stopnia n w postaci Newtona $N_n(x)$ w punkcie $x = t$ za pomocą uogólnionego algorytmu Hornera, w czasie $O(n)$.

2.2 Rozwiązanie

Uogólniony algorytm Hornera dla postaci Newtona pozwala obliczyć wartość wielomianu interpolacyjnego bez konieczności przekształcania go do postaci naturalnej. Wykorzystuje on rekurencyjną postać wielomianu Newtona:

$$N_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) \\ + \dots + f[x_0, \dots, x_n](x - x_0) \dots (x - x_{n-1})$$

Algorytm polega na obliczaniu wartości wielomianu od końca:

$$w_n := f[x_0, \dots, x_n] \\ w_k := f[x_0, \dots, x_k] + (x - x_k) \cdot w_{k+1} \quad \text{dla } k = n-1, n-2, \dots, 0 \\ N_n(x) := w_0$$

Algorithm 2: warNewton(x::VectorFloat64, fx::VectorFloat64, t::Float64)

```
1 nt ← fx[n]
2 for k ← n - 1 downto 0 do
3   | nt ← fx[k] + (t - x[k]) · nt
4 end
5 return nt
```

3 Zadanie 3

3.1 Cel zadania

Celem zadania jest przekształcenie wielomianu interpolacyjnego zapisanego w postaci Newtona:

$$P(x) = c_0 + c_1(x - x_0) + c_2(x - x_0)(x - x_1) + \dots + c_n(x - x_0) \dots (x - x_{n-1}),$$

gdzie współczynniki $c_0, c_1, \dots, c_n$ są ilorazami różnicowymi, na postać naturalną:

$$P(x) = a_0 + a_1x + a_2x^2 + \dots + a_nx^n.$$

3.2 Rozwiązanie

Celem algorytmu jest przekształcenie współczynników postaci Newtona fx (ilorazów różnicowych) na współczynniki postaci naturalnej a wielomianu.

Najwyższy współczynnik postaci naturalnej jest prosty do ustalenia:

$$a_n = fx_n,$$

dlatego ustawiamy go jako pierwszy.

Następnie przechodzimy od końca do początku współczynników Newtona. Każdy krok odpowiada „dopisywaniu” kolejnego ilorazu różnicowego do już zbudowanego wielomianu. Wielomian Newtona ma postać:

$$P(x) = fx_1 + fx_2(x - x_0) + fx_3(x - x_0)(x - x_1) + \dots,$$

co oznacza, że w każdym kroku trzeba przemnożyć aktualny wielomian przez $(x - x_i)$, a następnie dodać współczynnik $fx[i]$.

Pierwsza instrukcja w pętli:

$$a[i] = fx[i] - x[i] \cdot a[i + 1]$$

oblicza nowy współczynnik niższego stopnia po dodaniu członu Newtona.

Druga pętla:

$$a[j] \leftarrow a[j] - x[i] \cdot a[j + 1]$$

aktualizuje wszystkie wyższe współczynniki, ponieważ odpowiada przemnożeniu wielomianu przez czynnik $(x - x_i)$.

Algorithm 3: naturalna($x::\text{VectorFloat64}$, $fx::\text{VectorFloat64}$)

```
1  $a \leftarrow [0, 0, \dots, 0]$  długości  $n + 1$  ;
2  $a[n] \leftarrow c_n$ 
3 for  $i \leftarrow n - 1$  downto 0 do
4    $a[i] \leftarrow c_i - x_i \cdot a[i + 1]$  ;
5   for  $j \leftarrow i + 1$  to  $n - 1$  do
6      $a[j] \leftarrow a[j] - x_i \cdot a[j + 1]$  ;
7   end
8 end
9 return  $a$ 
```

4 Zadanie 4

4.1 Cel zadania

Celem zadania jest napisanie funkcji, która dokonuje interpolacji danej funkcji $f(x)$ w przedziale $[a, b]$ za pomocą wielomianu interpolacyjnego stopnia n w postaci Newtona. Funkcja powinna narysować zarówno interpolowaną funkcję $f(x)$, jak i jej wielomian Newtona $N_n(x)$ w przedziale $[a, b]$.

4.2 Rozwiązanie

Funkcja najpierw generuje węzły interpolacji zależnie od wybranej opcji. Dla węzłów równoodległych stosujemy:

$$x_k = a + kh, \quad h = \frac{b-a}{n}.$$

Dla węzłów Czebyszewa używamy wzoru:

$$x_k = \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{2k+1}{2n+2}\pi\right).$$

Następnie obliczamy wartości funkcji:

$$y_k = f(x_k),$$

i wyznaczamy ilorazy różnicowe wywołując funkcję `ilorazyRoznicowe(x, y)`.

Aby obliczyć wartość wielomianu interpolacyjnego Newtona w dowolnym punkcie t , korzystamy z funkcji `warNewton` zaimplementowanej wcześniej.

W ostatniej części generujemy siatkę $(100 \cdot (n+1))$ punktów w przedziale $[a, b]$ i rysujemy:

- wykres funkcji $f(x)$ i wykres wielomianu interpolacyjnego $N_n(x)$.

Algorithm 4: `rysujNnfx(f, a, b, n; wezly)`

```
1 if wezly = :rownoodlegle then
2   |  $h \leftarrow \frac{b-a}{n}$ 
3   |  $x \leftarrow [a + kh \mid k = 0, 1, \dots, n]$ 
4 else if wezly = :czebyszew then
5   |  $x \leftarrow \left[ \frac{a+b}{2} + \frac{b-a}{2} \cos\left(\frac{(2k+1)\pi}{2n+2}\right) \mid k = 0, 1, \dots, n \right]$ 
6 else
7   | throw(ArgumentError)
8 end

9  $y \leftarrow [f(x_k) \mid k = 0, 1, \dots, n]$ 
10  $fx \leftarrow \text{ilorazyRoznicowe}(x, y)$ 

11 Function Nn(t):
12   | return warNewton(x, fx, t)
13 end

14  $t \leftarrow \text{range}(a, b, \text{length} = 100 \cdot (n + 1))$ 
15  $ft \leftarrow [f(t_i) \mid i = 1, \dots, \text{length}(t)]$ 
16  $Nt \leftarrow [Nn(t_i) \mid i = 1, \dots, \text{length}(t)]$ 

17 plot(t, ft, label = "f(x)")
18 plot!(t, Nt, label = "N_n(x)")
```

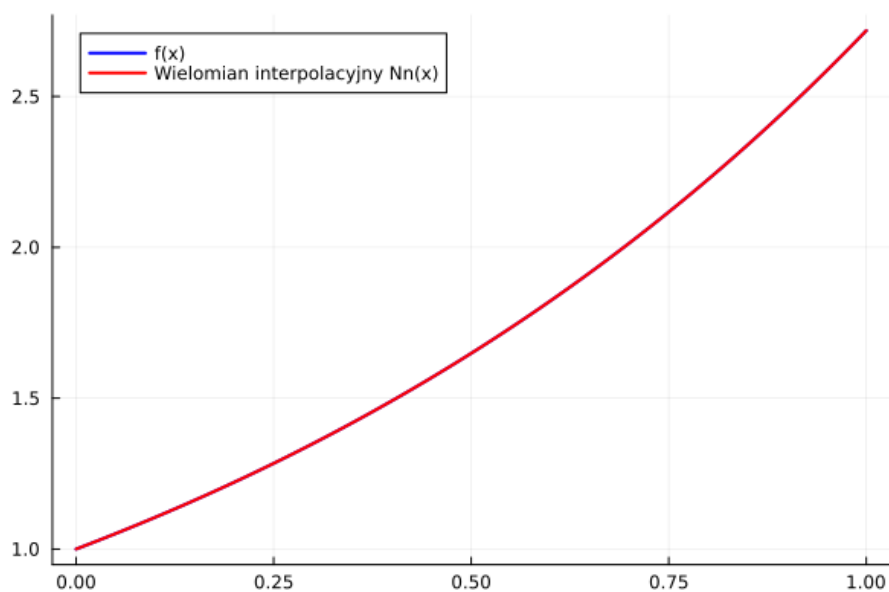
5 Zadanie 5

5.1 Cel zadania

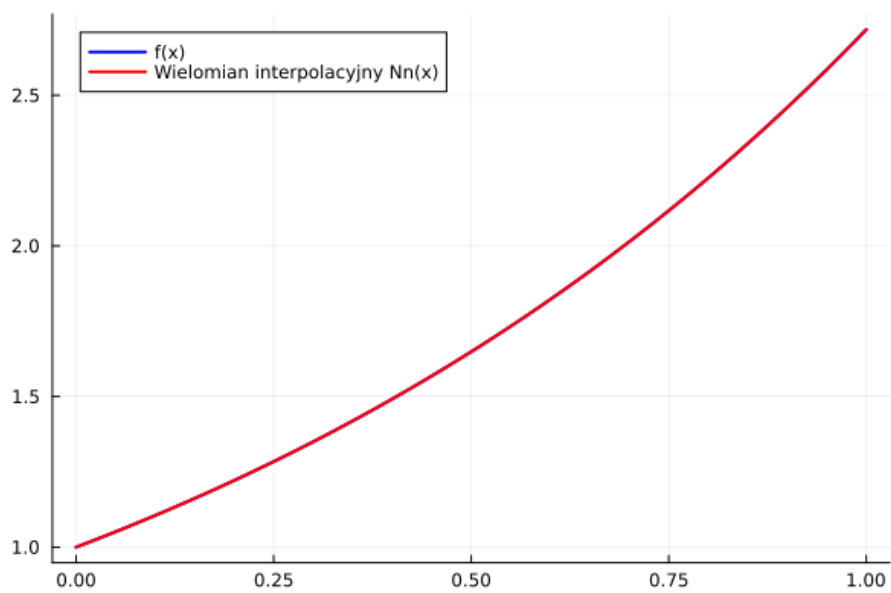
Wykorzystanie napisanej funkcji `rysujNnfx` dla $f_1(x) = e^x$ na przedziale $[0, 1]$ oraz $f_2(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$

5.2 Wykresy

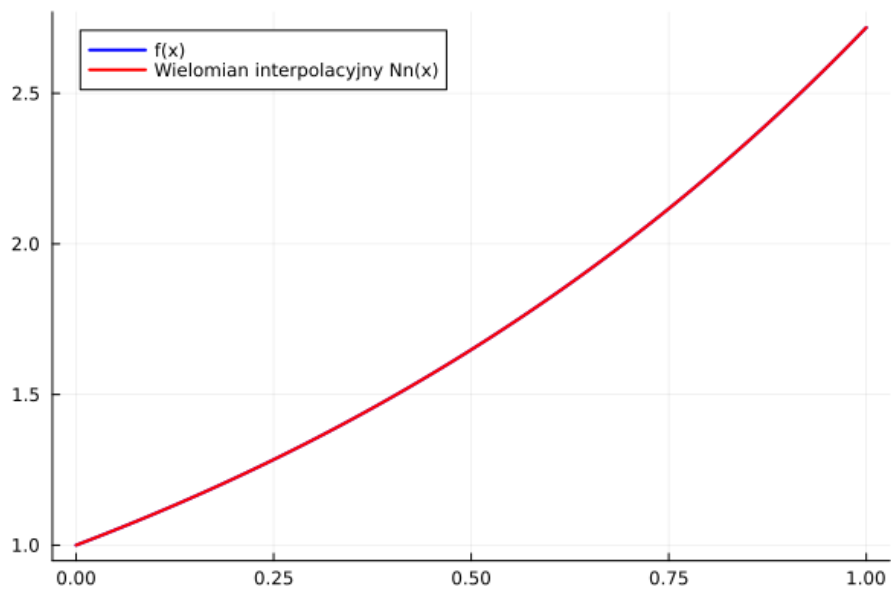
Funkcja $f_1(x) = e^x$ na przedziale $[0, 1]$



Rysunek 1: Interpolacja funkcji e^x dla $n = 5$ z węzłami równoodległymi

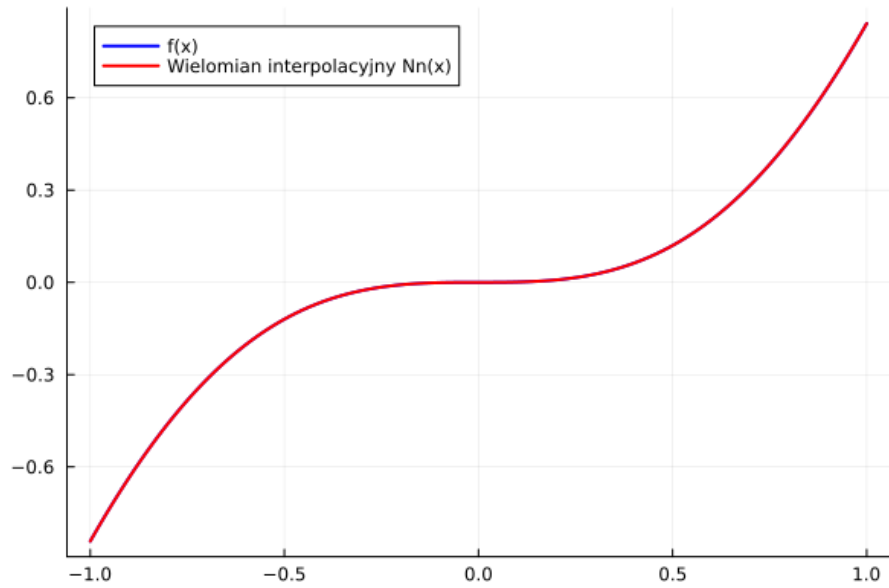


Rysunek 2: Interpolacja funkcji e^x dla $n = 10$ z węzłami równoodległymi

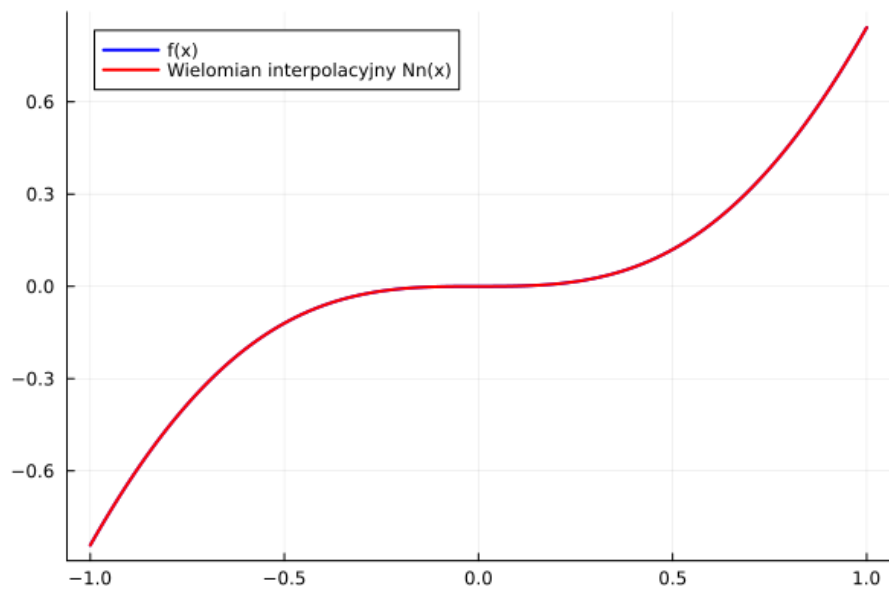


Rysunek 3: Interpolacja funkcji e^x dla $n = 15$ z węzłami równoodległymi

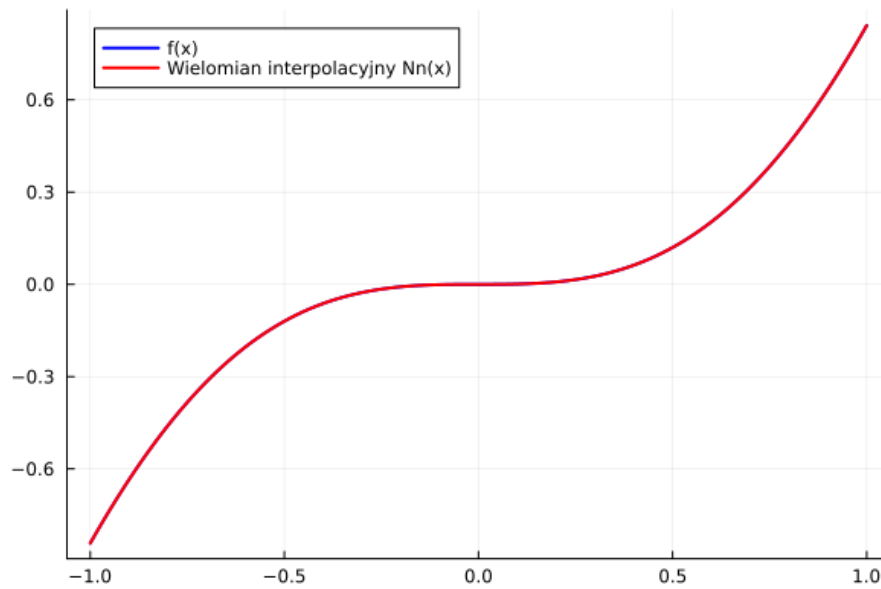
Funkcja $f_2(x) = x^2 \sin(x)$ na przedziale $[-1, 1]$



Rysunek 4: Interpolacja funkcji $x^2 \sin(x)$ dla $n = 5$ z węzłami równoodległymi



Rysunek 5: Interpolacja funkcji $x^2 \sin(x)$ dla $n = 10$ z węzłami równoodległymi



Rysunek 6: Interpolacja funkcji $x^2 \sin(x)$ dla $n = 15$ z węzłami równoodległymi

5.3 Interpretacja i wnioski

- Dla funkcji e^x i $x^2 \sin(x)$ interpolacja działa bardzo dobrze (wykresy nachodzą na siebie) nawet dla niewielkich n . Co pokazuje, że te funkcje można łatwo zinterpolować.

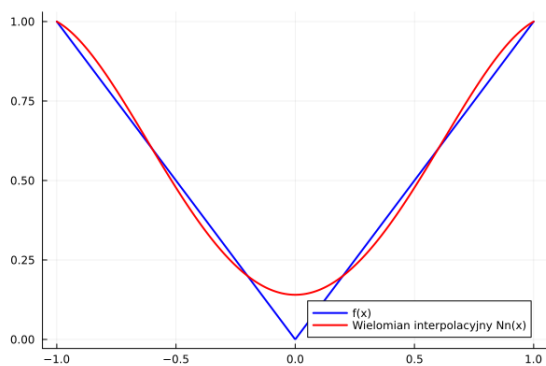
6 Zadanie 6

6.1 Cel zadania

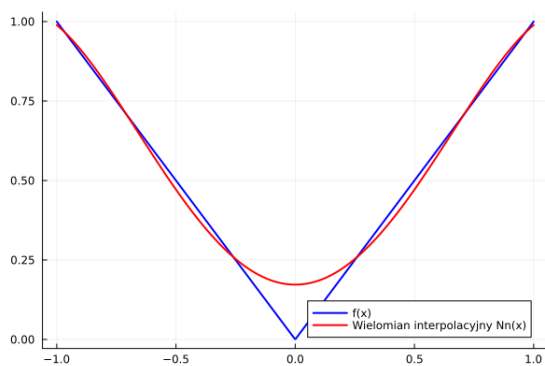
Wykorzystanie napisanej funkcji `rysujNnfx` dla $f_1(x) = |x|$ na przedziale $[-1, 1]$ oraz $f_2(x) = \frac{1}{1+x^2}$ na przedziale $[-5, 5]$

6.2 Wykresy

Funkcja $f_1(x) = |x|$ na przedziale $[-1, 1]$

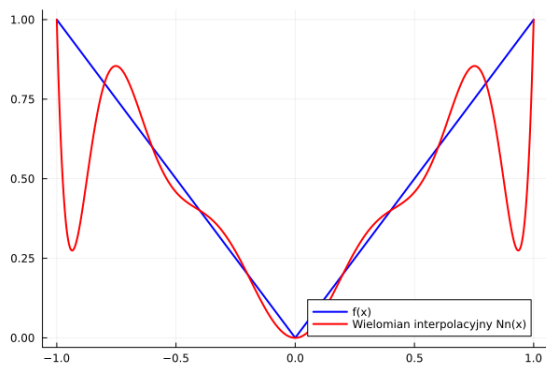


(a) $n = 5$, węzły równoodległe

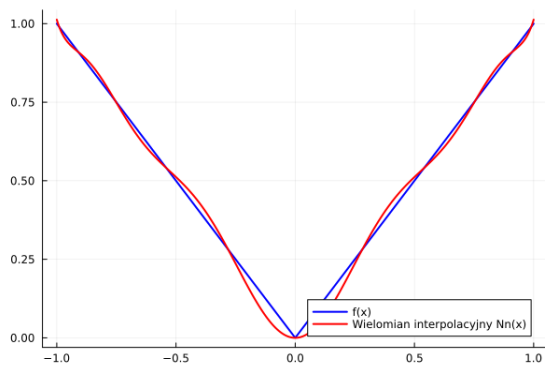


(b) $n = 5$, węzły Czebyszewa

Rysunek 7: Interpolacja funkcji $|x|$ dla $n = 5$

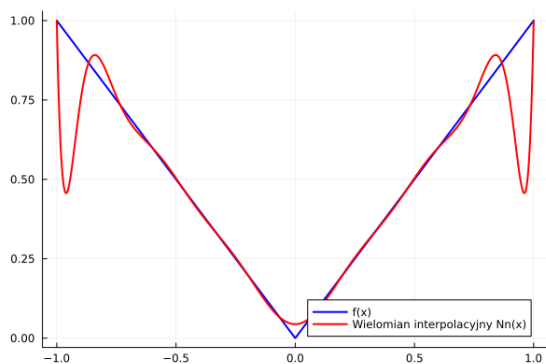


(a) $n = 10$, węzły równoodległe

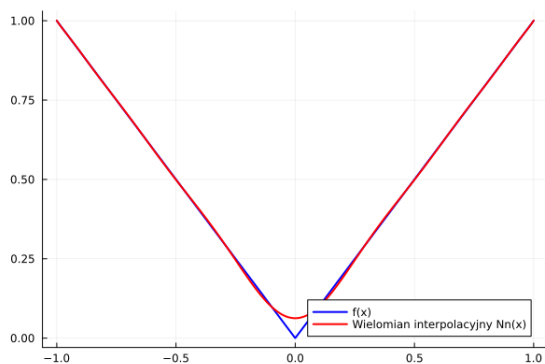


(b) $n = 10$, węzły Czebyszewa

Rysunek 8: Interpolacja funkcji $|x|$ dla $n = 10$



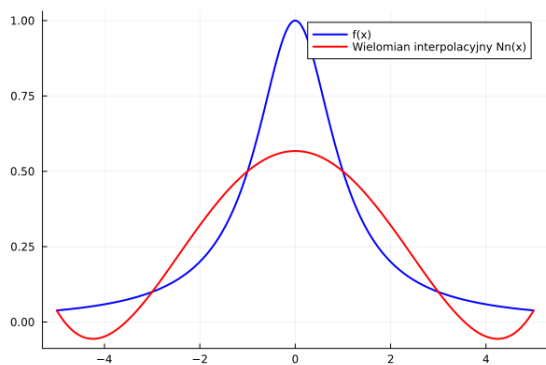
(a) $n = 15$, węzły równoodległe



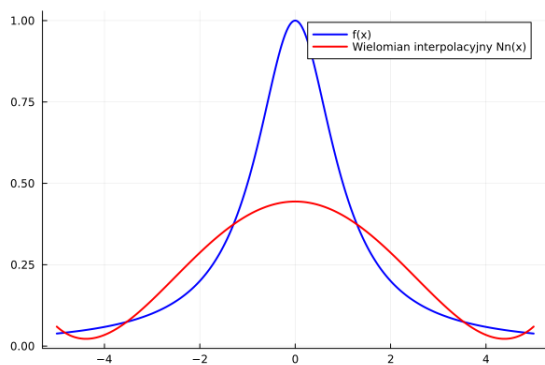
(b) $n = 15$, węzły Czebyszewa

Rysunek 9: Interpolacja funkcji $|x|$ dla $n = 15$

Funkcja $f_2(x) = \frac{1}{1+x^2}$ na przedziale $[-5, 5]$

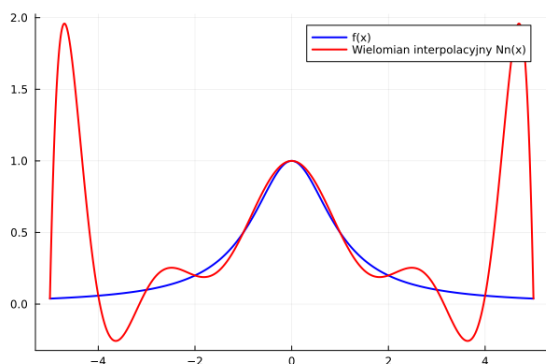


(a) $n = 5$, węzły równoodległe

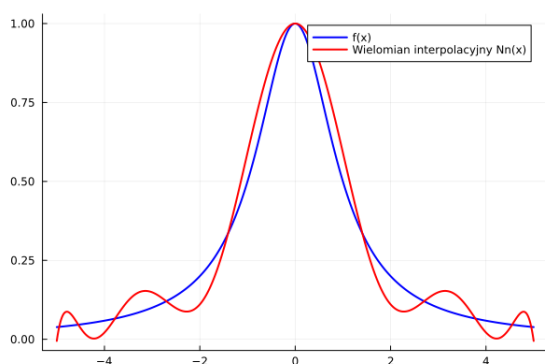


(b) $n = 5$, węzły Czebyszewa

Rysunek 10: Interpolacja funkcji $f_2(x) = \frac{1}{1+x^2}$ dla $n = 5$

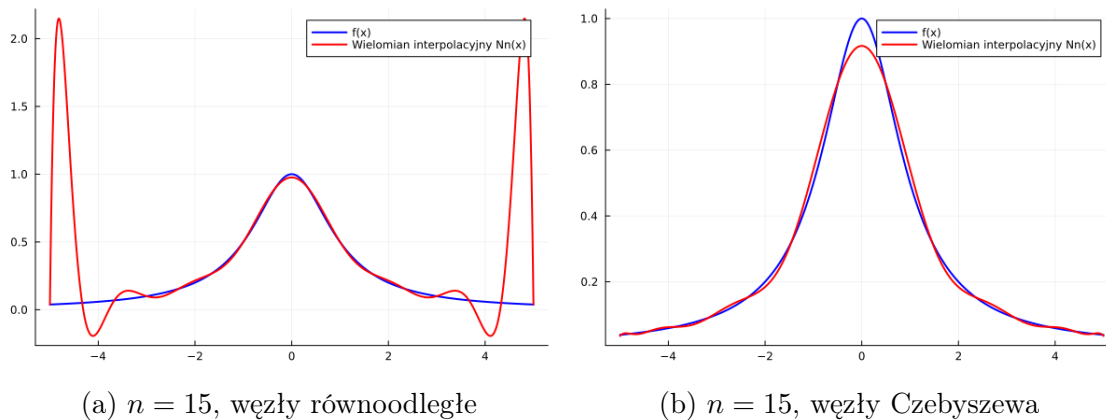


(a) $n = 10$, węzły równoodległe



(b) $n = 10$, węzły Czebyszewa

Rysunek 11: Interpolacja funkcji $f_2(x) = \frac{1}{1+x^2}$ dla $n = 10$



Rysunek 12: Interpolacja funkcji $f_2(x) = \frac{1}{1+x^2}$ dla $n = 15$

6.3 Interpretacja i wnioski

- Zwiększanie stopnia wielomianu interpolacyjnego dla węzłów równoodległych nie zawsze poprawia dokładność interpolacji. W przypadku obu funkcji obserwujemy, że wraz ze wzrostem stopnia wielomianu, interpolacja staje się coraz mniej dokładna na końcach przedziału.
- Dla funkcji $|x|$ w punkcie $x = 0$ funkcja ta nie ma pochodnej. Chcąc ją przybliżyć za pomocą gładkiego wielomianu powstaje błąd.
- Dla funkcji $\frac{1}{1+x^2}$ możemy zauważyć efekt Rungego - dla węzłów równoodległych błąd interpolacji rośnie na końcach przedziału wraz ze wzrostem n . Wynika to ze wzoru na błąd interpolacji:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \cdot (x - x_0)(x - x_1) \cdots (x - x_n).$$

Gdzie:

- $f(x)$ – funkcja, którą interpolujemy
- $P_n(x)$ – wielomian interpolacyjny stopnia n
- $f^{(n+1)}(\xi_x)$ – $(n+1)$ -sza pochodna funkcji f w pewnym punkcie ξ_x (zależnym od x), leżącym w przedziale $[a, b]$

Kluczowe są tu dwa czynniki działające jednocześnie:

1. Pochodne wyższych rzędów $f^{(n+1)}$ funkcji $\frac{1}{1+x^2}$ rosną bardzo szybko z n (rzędu $n!$)
2. Dla równoodległych węzłów iloczyn $(x - x_0) \cdots (x - x_n)$ osiąga duże wartości na krańcach przedziału

Mnożenie tych dwóch dużych wielkości daje wzrost błędu na końcach przedziału.

- Węzły Czebyszewa eliminują efekt Rungego dla funkcji $\frac{1}{1+x^2}$ – interpolacja pozostaje stabilna nawet dla dużych n . Są one gęściej rozłożone na końcach przedziału, co minimalizuje widoczny na krańcach błąd. Dla obu badanych funkcji węzły Czebyszewa dają lepsze przybliżenia niż węzły równoodległe.